

Ingest Payload Contract v1.2

Version	1.2 (supersedes 1.1)
Date	August 2026
Author	Maria Azevedo
Status	For implementation. Reconciled against the deployed endpoint, 2026-08-24

The wire format for delivering one certification extraction into Cartograph.

This version was verified against the Cartograph repository: model validations, strong-parameter lists, and the migration history. It corrects several v1.0 assumptions that would have failed at runtime. Those corrections are listed in Section 14.

Delivered to `POST /webhooks/runpod_ocr_results`. Endpoint and authentication are defined in *Integration Endpoints and Authentication*. A fully populated example accompanies this document as *Sample Ingest Payload*.

1. Three decisions this contract makes

The payload speaks Cartograph's vocabulary, not the engine's. Translation happens engine-side before sending. Cartograph validates `income_type` and `asset_type` against fixed lists, so the receiving service should reject an unknown value rather than guess. This keeps the Rails service thin and places the mapping on the side that holds the domain knowledge.

Members are addressed by client-assigned refs, never by name. Each member carries a ref invented by the engine ("`m01`"); `income`, `asset`, and `expense` records reference `member_ref`. The importer builds a `ref` → ID map as it creates members. Name matching across free-text strings is a known source of silent mis-attribution; addressing members by ref removes it.

Data records are full-replace; findings are not. Extraction-owned child records are cleared and rewritten on re-import. Findings are matched on their stable `finding_key` and updated in place, so a re-audit does not duplicate them and reviewer resolutions survive. See Section 11.

2. Envelope

```
{
  "schema_version": "1.2",
  "extraction_id": "ext_01JQ4X...", // echoed back in the result callback
  "engine_version": "idp-2.5.0",
  "extracted_at": "2026-08-17T14:22:09Z",

  "case_ref": "CAS600142", // REQUIRED - Cartograph Job#ref_number
  "target": {
    "job_id": 4821, // optional, if known
    "community_id": 67, // optional cross-check; 409 on mismatch
    "unit_number": "2-114"
  },

  "cert_review": { ... }, // Section 3
  "household_members": [ ... ], // Section 4
  "income_records": [ ... ], // Section 5
  "asset_records": [ ... ], // Section 6
  "expense_records": [ ... ], // Section 7
  "findings": [ ... ] // Section 8 - carries item_code
}
```

```
}
```

case_ref is the only required routing field: Cartograph's controller resolves it with `Job.find_by(ref_number: case_ref)`. An unresolvable case_ref is recorded on the scan and reported back through the result callback rather than rejected outright. Cartograph derives the cert_review from the job; there is no create route, and one is auto-created on first access.

Deliberately excluded: per-page OCR provenance, page classification, and document grouping. These are engine diagnostics with no operational use in Cartograph; they stay engine-side and are linked by extraction_id. Income calculations are also excluded; see Section 12.

3. cert_review

Scalar fields only. Omit any key with no value; do not send null, which would blank a staff-entered value.

```
{
  "cert_type":      "annual",
  "effective_date": "2026-07-01",
  "move_in_date":   "2019-03-15",
  "unit_number":    "2-114",
  "hh_size":        2,
  "annual_income":  "46377.18",
  "annual_assets":  "3362.55",
  "head_of_household_name": "Marcus Halvorsen",
  "tenant_rent":     "865.00",
  "gross_rent":      "940.00",
  "utility_allowance": "75.00",
  "max_program_rent": "1024.00"
}
```

cert_type accepts initial, annual, interim only. The engine's four certification types map as MI → initial, AR → annual, IR → interim; **AR-SC has no target value** and is an open decision (Section 15).

result and signed_off_at are staff state and are never written by the importer.

4. household_members

ref is required and must be unique within the payload. first_name and last_name are validated for presence.

```
{
  "ref":      "m01",
  "first_name": "Marcus",
  "last_name": "Halvorsen",
  "date_of_birth": "1974-11-02", // ISO; parsed engine-side
  "relationship": "Head of Household",
  "is_hoh":      true,
  "is_disabled": false,
  "full_time_student": false,
  "ssn_last4":   "3168", // last four only, see Section 13
  "email":       "m.halvorsen@example.com",
  "cell_phone":  "555-0142",
  "sort_order":  1
}
```

The engine does not extract relationship-to-head. It sends "Head of Household" where the head flag is set and omits the key otherwise, leaving the field unset rather than inferring a value.

5. income_records

One object per **(member, source)** pair. The engine's flat per-paystub output is grouped into these parents before sending; paystubs, VOIs, and zero-income affidavits nest inside.

```
{
  "member_ref": "m01",
  "income_type": "non_federal_wages", // validated, Section 10.1
  "source_name": "Northgate Logistics LLC",
  "verification_status": "received",
  "frequency_of_pay": "biweekly",
  "date_received": "2026-06-18",
  "employment_start_date": "2019-04-01",
  "employer_address": "1400 Depot Rd, Salem, OR 97301",
  "self_declared_amount": "45900.00",
  "source_of_declaration": "Application",

  "paystubs": [
    { "pay_date": "2026-06-05", "gross_pay": "1742.10",
      "ytd_amount": "21071.75", "pay_frequency": "biweekly" }
  ],
  "vois": [
    { "voi_type": "Employer Verification", "date_received": "2026-06-18",
      "rate_of_pay": "21.78", "hours_per_pay_period": "80.00",
      "frequency_of_pay": "biweekly", "ytd_amount": "18406.00",
      "ytd_start_date": "2026-01-01", "ytd_end_date": "2026-06-05" }
  ],
  "zero_income": null
}
```

Zero income is a child, not a type. zero_income is not a valid income_type and fails validation. Set the parent to other and attach a zero_income object with affidavit_date, signed, notes.

6. asset_records

```
{
  "member_ref": "m01",
  "asset_type": "checking", // validated, Section 10.2
  "institution_name": "Cascade Credit Union",
  "current_value": "3234.11",
  "bank_stmt_avg_balance": "2988.40",
  "annual_income_from_assets": "12.40",
  "interest_rate": "0.004000",
  "percentage_of_ownership": "100",
  "verification_status": "received",

  "bank_statements": [
    { "statement_date": "2026-05-31", "balance": "3104.02",
      "source_label": "May statement" }
  ],
  "voa": { "voa_date": "2026-06-11", "reported_value": "3234.11",
    "source": "Cascade Credit Union",
    "month_1_balance": "2910.00", "month_2_balance": "3050.40", // only when the VOA
    "month_3_balance": "2875.12", "month_4_balance": "3104.02", // lists monthly balances
    "month_5_balance": "2990.00", "month_6_balance": "3234.11" } // month 1 = oldest
  }
}
```

month_1_balance ... month_6_balance are sent only when the verification of assets lists individual monthly balances instead of a six-month average (Chase and some others). Month 1 is the oldest; fewer than six months fills from month 1; more than six sends the most recent six. When they are sent, bank_stmt_avg_balance carries the mean of the months sent, so it agrees with the average Cartograph computes from them.

Never send self_declared_balance. It appears in the controller's permitted parameters but has **no backing column**. It is a seeded calculation-method code. Because records are built by mass assignment, including it raises ActiveRecord::UnknownAttributeError and returns a 500. Use manual_balance for a declared balance

and self_declared_income for declared asset income.

6b. checklist_matches

One entry per checklist row of the case the packet can answer, keyed on the row's own finding id from the case request (checklist_rows[], or requirements[].community_defaults.checklist_items[].case_finding_ids). Rows the engine cannot map to a document type it knows are left out and stay untouched.

```
{ "finding_id": 88412, "found": true, "pages": [2, 3], "confidence": 0.85,
  "note": "[Scan] HUD 50059 present, pages 2-3. Not signed: the signature lines are blank." }
{ "finding_id": 88419, "found": true, "pages": [38, 39], "confidence": 0.9,
  "note": "[Scan] HUD 9887 present, pages 38-39. Signed by Yolanda Bribiesca on 2026-09-16." }
{ "finding_id": 88421, "found": false, "pages": [], "confidence": 0.9,
  "note": "[Scan] No HUD Race and Ethnic Data Form in the packet." }
```

found: false means the engine looked and the form is not in the packet. A row about a property of a form ("HUD 50059 Includes the Correct Income Limits") is not answered by the form being present; such rows are left out and the engine's findings cover them. A row asking for the previous form ("Previous HUD 50059") matches the previous certification the packet carries, which the audit otherwise sets aside. A form the engine knows to be incomplete says so in its note. confidence is how sure the engine is of the match and the note together: 0.9 when the row and the document share a form number, 0.8 when the document's title is printed in the row's label, 0.6 when only words match; lowered when the classifier placed the page by nearest match or a signature could only be inferred. The note is written for the row's note field and always starts with [Scan] : on a later run Cartograph clears a note with that prefix from any row it sent that the reply omits, and never touches a note staff typed. The status stays Cartograph's to set.

7. expense_records

```
{ "member_ref": "m03", "expense_type": "dependent_care",
  "provider_name": "Bright Start Childcare", "annual_amount": "6240.00" }
```

Only these four keys are safe. monthly_amount and verification_status are permitted by the controller but have no columns and will 500 the request.

The engine does not currently extract expense data; this section is defined for completeness and will arrive empty until that capability exists (Section 15).

8. Checklist determinations ride on findings

Cartograph dropped cert_review_checklist_items on 2026-08-18, replacing it with CertChecklistTemplate (the per-program item definitions, now carrying item_code and cert_type_scope) plus CertReviewFinding (the per-case result). There is no longer a separate table of per-review checklist rows, so this payload no longer carries a separate checklist_items array.

Instead, a finding that answers a checklist item carries that item's item_code. One finding per answered item; findings not tied to an item omit the key.

This requires one column that does not exist yet: item_code on cert_review_findings. item_code was added to the *template* table, not to findings, so a finding currently has no way to reference the item it answers. See the *Schema Change Request*, which lists this as the highest-priority addition.

Item codes are per-program and editable through the community funding program admin screen, so the engine cannot hardcode them. They must be supplied, either as an export or through the trigger payload. An item_code

the engine does not recognise is left unanswered rather than guessed.

9. findings

Findings land in `cert_review_findings`, which already models subject, category, result, and assignment.

```
{
  "finding_key": "INCOME_YTD_MISMATCH:m01:northgate", // stable across re-imports
  "item_code": "INC_FOUR_PLUS_PAYSTUBS", // omit when not a checklist item
  "category": "income", // unit_rent | household_member | income |
  // asset | expense | file_review
  "subject_ref": { "type": "income_record", "member_ref": "m01",
    "source_name": "Northgate Logistics LLC" },
  "label": "Paystub YTD does not match VOI YTD",
  "result": "non_compliant", // compliant | non_compliant | na
  "assignment": "client", // internal | client | procedural_issue
  "correction_required": "Obtain corrected VOI or a fourth paystub for this source",
  "confidence": 0.82,
  "resolution_type": "recalculation", // presence_only | recalculation
  "pages": [12, 13, 14],
  "position": 3
}
```

`subject_ref` is resolved by the importer to `subject_type` / `subject_id` against records created earlier in the same transaction. A finding with no subject (type: "case") becomes a `file_review` finding with a null subject, matching Cartograph's existing convention.

`finding_key` is deterministic, derived from the finding type plus the member and source it concerns, so a re-import matches an existing finding rather than duplicating it. **Findings whose key and label are unchanged retain their existing result and any reviewer resolution.** Only findings whose determination changed are reset.

`confidence`, `resolution_type`, and `pages` require the new columns listed in the *Schema Change Request*. They drive auto-approve gating and reviewer guidance respectively.

10. Controlled vocabularies

Two are enforced by model validation; the rest are convention but should still be normalized so filters behave.

10.1 income_type (validated)

Engine value	Cartograph value
Non-Federal Wage	<code>non_federal_wages</code>
Federal Wage	<code>employment</code> (closest available; see note below)
Social Security	<code>social_security</code>
Supplemental Security Income	<code>ssi</code>
Social Security Disability	<code>social_security</code>
Pension	<code>pension</code>
Temporary Assistance	<code>public_assistance</code>
Child Support	<code>child_support</code>
Self-Employment	<code>self_employment</code>
Zero Income	<code>other</code> (+ <code>zero_income</code> child)
Other Income	<code>other</code>

No federal-specific value exists, so federal wages map to the generic employment. Confirm this is the intended target before go-live.

Cartograph accepts nine further values the engine does not currently emit: `alimony`, `wages_and_salaries`, `income_in_cash`, `unemployment_benefits`, `armed_forces_pay`, `veterans_benefits`, `investments`, `real_estate_income`, `student_financial_aid`.

10.2 asset_type (validated)

Cartograph allows nine values: `checking`, `savings`, `money_market`, `cd`, `stocks`, `bonds`, `real_estate`, `retirement`, `other`.

Engine value	Cartograph value
Checking	<code>checking</code>
Savings	<code>savings</code>
CD	<code>cd</code>
Investment	<code>stocks</code>
Retirement	<code>retirement</code>
Real Estate	<code>real_estate</code>
Life Insurance, Cryptocurrency, Prepaid Card, Peer-to-Peer, ABLE Account, Cash, Annuity, Direct Express	<code>other</code>

Eight engine values collapse to `other`. The original is preserved in notes and reported in the result callback's warnings.

10.3 Remaining vocabularies

Field	Accepted values
<code>pay / income frequency</code>	<code>weekly</code> <code>biweekly</code> <code>semimonthly</code> <code>monthly</code> <code>annually</code>
<code>verification_status</code>	<code>pending</code> <code>requested</code> <code>received</code> <code>waived</code>
<code>expense_type</code>	<code>dependent_care</code> <code>disability_assistance</code> <code>medical</code> <code>other</code>
<code>finding result</code>	<code>compliant</code> <code>non_compliant</code> <code>na</code> <code>resolved</code>
<code>finding assignment</code>	<code>internal client</code> <code>procedural_issue</code>
<code>checklist result</code>	<code>pending</code> <code>pass</code> <code>fail</code> <code>na</code>
<code>issue severity</code>	<code>blocking</code> <code>warning</code> <code>info</code>
<code>relationship</code>	<code>Head of Household</code> · <code>Spouse</code> · <code>Co-Head</code> · <code>Minor Child</code> · <code>Foster Child/Adult</code> · <code>Live-in Aide</code> · <code>Other Adult Member</code>

11. Import order

The whole import runs in one transaction. Order matters because children carry foreign keys to records created earlier in the same request.

1. Verify signature and timestamp, before parsing the body.

2. Resolve the job, then find-or-create its cert review. A missing job is a 409: the packet is valid, the target is not ready.
3. Open the transaction and clear prior **extraction-owned** child records (see below).
4. Update cert_review scalars, using only keys present in the payload. Leave result, signed_off_at, and notes untouched.
5. Create household members, building the ref → ID map.
6. Create income records, then their paystub / VOI / zero-income children. An unresolvable member_ref is a hard failure, not a null.
7. Create asset records, then bank statements and VOA. Then expense records.
8. Upsert findings by finding_key, carrying item_code where the finding answers a checklist item.
9. Commit, then run CertIncomeCalculationService. Running after commit ensures it reads settled data.
10. Post the result callback.

"Extraction-owned" requires a marker

Full-replace in step 3 is only safe if the delete can distinguish records this endpoint created from records an analyst entered by hand. Cartograph has no such marker today. Blind-deleting every cert_income_record on re-import would destroy reviewer work.

The *Schema Change Request* asks for a source column defaulting to "manual", so the delete becomes `where(source: "extraction")`. **Until that exists, treat the endpoint as create-only against an empty cert review and reject a second import with 409.**

12. Income calculations stay out of the payload

Cartograph computes its own into cert_income_calculations, and its method list (self_declared, paystub_avg, paystub_max, paystub_ytd, voi, voi_ytd) does not align with the engine's four internal methods. The engine sends raw paystubs and VOIs and lets Cartograph calculate.

The engine still computes income internally, since that is how it produces findings. Where the two diverge materially, the engine raises the divergence as a finding rather than writing a competing calculation row.

13. SSN handling

This contract carries ssn_last4 and no full SSN field, deliberately. Cartograph's ssn column is plain text with no Active Record encryption declared in the model layer. Sending full digits would move them from a masked, controlled store into an unencrypted one.

If full write-back is needed later, the precondition is encrypts :ssn on CertHouseholdMember plus a backfill; the field would be added at v1.2, not before.

14. Corrections from earlier versions

Rows 1 to 6 were corrected in v1.1 against the repository; rows 7 to 9 in v1.2 against the deployed endpoint.

#	v1.0 assumption	Correct behaviour
1	Findings go to cert_review_issues	cert_review_findings, which models polymorphic subject, category, result enum, and assignment
2	Checklist items are created/replaced	Auto-seeded by after_create :seed_checklist; upsert by item_code, unique index on (cert_review_id, item_code)
3	Severity values warning / error / info	blocking / warning / info. error fails validation
4	self_declared_balance is a column	It is a seeded calculation-method code, not a column. self_declared_income and manual_balance are the real columns
5	percentage_of_ownership and real-estate values have no column	All three exist on cert_asset_records
6	cert_type mapping complete	Only three values exist; AR-SC unmapped
7	Checklist determinations upsert into cert_review_checklist_items	That table was dropped 2026-08-18. Determinations ride on findings via item_code, which findings do not yet have
8	Signature is v1, <base64>, headers X-Idp-*	Deployed endpoint expects a hex digest with no prefix, on X-Timestamp / X-Signature
9	Routing on target.job_id	Controller resolves case_ref against Job#ref_number

15. Open decisions

1. **item_code list.** Codes are per-program and editable, so the engine cannot hardcode them. Supply them as an export or in the trigger payload.
2. **AR-SC.** cert_type has three values and the engine handles four. Add a value, or map AR-SC onto annual with the distinction carried elsewhere?
3. **Packet definition.** Which attachment set constitutes the packet: JobDocument#document filtered by document_class/source, or CertReview#documents?
4. **Expense extraction.** Expenses affect adjusted income on HUD certifications. Should the engine extract them, or do they remain staff-entered?
5. **Income authority.** Confirmed as Cartograph computing, with engine divergence surfaced as a finding. Recorded here so it is explicit rather than assumed.